# Project-Plan_GO-IT

# Социальная образовательная платформа "GO-IT: социальный код"

## Концепция и технический план демонстрационного MVP

- **Версия:** 0.1
- **Команда:** 2 студента-первокурсника направления ИСиТ
- **Срок:** 6-8 недель
- **Формат:** адаптивная веб-платформа
- **Основной стек:** Go, PostgreSQL, `html/template`, HTMX, CSS, Docker Compose.

## 1. Резюме проекта

GO-IT - социально значимая образовательная веб-платформа для школьников и студентов, которая помогает познакомиться с IT-профессиями через практические сценарии, выбрать последовательность обучения и найти возможности для взаимопомощи.

Платформа объединяет:

- **«Первый кейс»** - интерактивную мини-симуляцию задач Frontend-разработчика, Backend-разработчика или IT Project Manager.
- **P2P-менторство** - каталог наставников-студентов и форму заявки на консультацию.
- **«Цифровой волонтёр»** - каталог небольших цифровых задач от социальных и образовательных организаций.
- **Образовательные материалы** - дорожные карты (roadmaps), несколько статей блога и демонстрационный раздел новостей.
- **Доступный интерфейс** - светлая и тёмная темы, режим повышенной контрастности, адаптивная вёрстка.

## 2. Проблема, цель и социальная значимость

### Проблема

Начинающим сложно разобраться в разнообразии IT-специальностей, понять характер повседневных задач и составить последовательный план обучения. Материалы разбросаны по разным источникам, а описания профессий не всегда позволяют представить реальную работу. Одновременно более опытные студенты могут делиться знаниями, а небольшим социальным организациям иногда нужна посильная цифровая помощь.

### Решение

GO-IT объединяет знакомство с профессиями, бесплатные учебные маршруты и демонстрационные инструменты взаимопомощи в одном интерфейсе.

### Социальная значимость

- снижение барьера входа в IT за счёт бесплатных структурированных материалов;
- знакомство с профессиями через практические ситуации, а не только описания;
- развитие культуры взаимного обучения между студентами (P2P - равный равному);
- потенциальная связь цифровых навыков студентов с задачами социальных организаций;
- внимание к доступности интерфейса.

На этапе MVP социальный эффект следует называть **ожидаемым**. Для подтверждения реальной пользы потребуются тестирование на целевой аудитории и сбор обратной связи.

### Цель

За 6-8 недель разработать адаптивную веб-платформу на Go, в которой работает хотя бы один интерактивный профориентационный кейс, доступны дорожные карты и материалы, а каталоги менторов и волонтёрских задач демонстрируют сценарии взаимопомощи.

### Задачи

1. Описать аудиторию, проблему и пользовательские сценарии.
2. Спроектировать архитектуру, маршруты и минимальную модель данных.
3. Реализовать общий интерфейс и доступные темы.
4. Разработать кейс с серверной проверкой ответов и расчётом результата.
5. Добавить roadmaps и несколько статей.
6. Создать каталоги менторов и волонтёрских запросов.
7. Реализовать формы заявок с валидацией и сохранением в PostgreSQL.
8. Настроить Docker Compose, тесты и документацию.
9. Подготовить сценарий демонстрации.

## 3. Аудитории и сценарии

| Пользователь | Потребность | Функции GO-IT |
|---|---|---|
| **Школьник** | Понять, какие профессии существуют | Кейсы, описания направлений, roadmaps |
| **Студент младших курсов** | Выбрать специализацию | Материалы и практические сценарии |

| Пользователь | Потребность | Функции GO-IT |
|---|---|---|
| **Студент-наставник** | Помочь начинающим | Профиль и заявки на консультацию |
| **Студент-волонтёр** | Применить навыки на полезной задаче | Каталог запросов и форма отклика |
| **Социальная организация** | Найти помощь для небольшой цифровой задачи | Демонстрационная форма запроса |
| **Администратор** | Поддерживать контент и тестовые данные | Seed-данные; простая админка (только если останется время) |

### Сценарий «Первый кейс»

1. Пользователь выбирает направление.
2. Читает вводную ситуацию и отвечает на несколько вопросов.
3. Go-сервер проверяет каждый ответ и определяет следующий шаг.
4. После завершения сервер рассчитывает результат.
5. Пользователь получает баллы, пояснения и ссылку на соответствующую дорожную карту.

Результат - ознакомительный, а не психологический диагноз и не окончательная рекомендация профессии.

### Сценарий P2P

Пользователь открывает каталог, выбирает наставника, заполняет форму и получает подтверждение. Заявка сохраняется в БД. В MVP не нужны чат, календарная синхронизация и автоматический подбор пары.

### Сценарий цифрового волонтёрства

Пользователь изучает задачу, требования к навыкам и ожидаемый результат, отправляет отклик, после чего сервер валидирует форму и сохраняет запись. MVP не претендует на полноценную систему управления проектами.

## 4. Границы MVP

### Реализовать обязательно (Must)

- адаптивную главную страницу и навигацию;
- каталог направлений;
- минимум одну содержательную дорожную карту;
- минимум две статьи;
- один полностью работающий интерактивный кейс;

- серверную проверку ответов и расчёт результата;
- каталог менторов с демонстрационными данными;
- каталог волонтёрских задач с демонстрационными данными;
- хотя бы одну работающую форму заявки с сохранением в БД;
- светлую и тёмную темы;
- обработку основных ошибок и пустых состояний;
- запуск через Docker Compose;
- тесты ключевой бизнес-логики.

### Желательно (Should)

- форму заявки и для ментора, и для волонтёрской задачи;
- режим повышенной контрастности;
- две-три дорожные карты;
- базовые HTTP-тесты;
- простой экран состояния заявки в локальном демонстрационном режиме.

### Не входит в MVP

- полноценная регистрация и восстановление пароля;
- встроенный чат и видеозвонки;
- календарь и автоматический подбор наставников;
- полный workflow обработки волонтёрских заявок;
- полноценная CMS и сложная админ-панель;
- автоматический импорт новостей;
- мобильные приложения;
- микросервисы, Kubernetes, Redis и очередь задач;
- Telegram-бот до завершения основного веб-сценария.

Если сроки начинают срываться, сокращать нужно объём контента и второстепенные функции, а не качество главного сценария.

## 5. Функциональные требования

| ID | Требование | Приоритет | Критерий приёмки |
|---|---|---|---|
| **FR-01** | Главная и навигация | Must | Все ключевые разделы доступны |
| **FR-02** | Каталог направлений | Must | Карточки ведут на страницы направлений |
| **FR-03** | Roadmap | Must | Этапы обучения показаны последовательно |

| ID | Требование | Приоритет | Критерий приёмки |
|---|---|---|---|
| **FR-04** | Блог | Must | Доступны минимум две отдельные статьи |
| **FR-05** | Прохождение кейса | Must | Ответы проверяются сервером |
| **FR-06** | Результат кейса | Must | Баллы и пояснения соответствуют ответам |
| **FR-07** | Каталог менторов | Should | Отображаются тестовые карточки |
| **FR-08** | Форма заявки | Should | Данные валидируются и сохраняются |
| **FR-09** | Каталог волонтёрских задач | Should | Есть описание задач и навыков |
| **FR-10** | Тема оформления | Must | Тема применяется на основных страницах |
| **FR-11** | Высокая контрастность | Should | Контраст и фокус заметно улучшены |
| **FR-12** | Docker Compose | Must | Проект запускается по README |
| **FR-13** | Тесты | Must | Ключевые правила бизнес-логики покрыты тестами |

# 6. Архитектурные решения

## Архитектурный стиль

Предлагается модульный монолит: одно Go-приложение, разделённое на пакеты по зонам ответственности. Для команды из двух человек это проще микросервисов: нет отдельного развёртывания каждого модуля, распределённого логирования и сетевых контрактов между сервисами. При этом границы модулей позволяют развивать систему постепенно.

## Общая схема

```
Браузер пользователя
         |
         | HTTP / HTTPS
         v
 Go web application
   ├── Router и middleware
   ├── Page handlers
   ├── Form handlers
   ├── Case service (логика симуляции)
```

```
    ├── Content service (roadmaps и блог)
    ├── Catalog service (менторы и волонтёрские задачи)
    ├── Application service (заявки)
    └── Repositories
          |
          +--------------------+
          |                    |
          v                    v
     PostgreSQL          Markdown / static
     заявки и данные     контент, CSS, JS,
     каталогов           изображения
```

## Почему серверный рендеринг

`html/template` генерирует HTML на стороне Go. HTMX можно использовать точечно для обновления отдельных фрагментов, но обычные HTML-формы с POST и redirect тоже подходят. Отдельный SPA на React/Vue не нужен: он удвоит объём работ и потребует отдельного API-контракта.

## Структура проекта

```
go-it/
  ├── cmd/web/main.go
  ├── internal/
  │   ├── app/
  │   ├── config/
  │   ├── http/
  │   │   ├── router.go
  │   │   ├── middleware/
  │   │   ├── handlers/
  │   │   └── render/
  │   ├── domain/
  │   ├── service/
  │   │   ├── case_service.go
  │   │   └── application_service.go
  │   ├── repository/
  │   │   └── postgres/
  │   └── session/
  ├── templates/
  │   ├── layouts/
  │   ├── pages/
  │   └── partials/
  ├── content/
  │   ├── roadmaps/
  │   └── blog/
  ├── static/css/
  ├── static/js/
  ├── migrations/
```

```
├── tests/
├── Dockerfile
├── docker-compose.yml
├── .env.example
├── Makefile
├── go.mod
└── README.md
```

# 7. Технологический стек

| Слой | Технология | Обоснование |
|---|---|---|
| **Backend** | Go | Основной язык и предмет демонстрации |
| **HTTP** | `net/http` + `chi` | Простой router и middleware |
| **HTML** | `html/template` | Серверный рендеринг с HTML-экранированием |
| **Динамика** | HTMX, выборочно | Обновление фрагментов без SPA |
| **Стили** | CSS + Custom Properties | Темы и адаптивность |
| **JavaScript** | Vanilla JS, минимально | Настройки темы и небольшие взаимодействия |
| **БД** | PostgreSQL | Хранение заявок и структурированных данных |
| **Доступ к БД** | `pgx` или `database/sql` | Явные SQL-запросы |
| **Миграции** | `golang-migrate` или аналог | Версионирование схемы |
| **Контент** | Markdown | Статьи и roadmaps в Git |
| **Тестирование** | `testing` , `httptest` | Unit- и HTTP-тесты |
| **Контейнеры** | Docker Compose | Повторяемый локальный запуск |
| **CI** | GitHub Actions/GitLab CI, опционально | Форматирование и тесты |

На текущем этапе не нужны Redis, Kubernetes, микросервисы, внешняя CMS, сложная авторизация и отдельный frontend build pipeline.

# 8. Модель данных

## Минимальные сущности

| Сущность | Назначение | Хранение |
|---|---|---|
| `directions` | Направления IT | Seed-данные или PostgreSQL |
| `roadmaps` | Метаданные дорожных карт | Markdown + метаданные |

| Сущность | Назначение | Хранение |
|---|---|---|
| `cases` | Описание кейсов | JSON/seed-данные |
| `case_questions` | Вопросы кейса | JSON/seed-данные |
| `case_options` | Варианты ответов и баллы | JSON/seed-данные |
| `mentors` | Демонстрационные профили | Seed-данные |
| `volunteer_requests` | Демонстрационные задачи | Seed-данные |
| `applications` | Заявки и отклики | PostgreSQL |

Статьи и текст roadmaps можно хранить в Markdown: для нескольких статических материалов админ-панель не оправдывает затраты.

## Пример SQL для заявок

```
CREATE TABLE applications (
    id BIGSERIAL PRIMARY KEY,
    kind TEXT NOT NULL
        CHECK (kind IN ('mentor', 'volunteer')),
    target_id TEXT NOT NULL,
    name TEXT NOT NULL,
    contact TEXT NOT NULL,
    message TEXT NOT NULL DEFAULT '',
    status TEXT NOT NULL DEFAULT 'new'
        CHECK (status IN ('new', 'reviewed', 'closed')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX idx_applications_kind_status
    ON applications (kind, status);
CREATE INDEX idx_applications_created_at
    ON applications (created_at DESC);
```

Это минимальный пример. Если каталоги менторов и задач будут храниться в PostgreSQL, `target_id` лучше заменить внешними ключами или нормализовать модель.

## Начальные данные

- 3 направления: Frontend, Backend, IT Project Management;
- 2-3 дорожные карты;
- 2 статьи;
- 3-5 карточек наставников;
- 3-5 волонтёрских запросов;
- 1 полноценный кейс.

# 9. Модуль «Первый кейс»

Это центральная функция, на которой демонстрируется реальная бизнес-логика Go.

## Варианты сценариев

- **Frontend:** выбрать способ исправить проблему с расположением элементов, состоянием кнопки или доступностью формы.
- **Backend:** определить корректную обработку ошибки или данные, которые следует проверить перед сохранением.
- **IT Project Manager:** выбрать следующий шаг, когда требования заказчика неполны, а команда по-разному понимает результат.

*Для MVP достаточно одного полноценного кейса. Остальные можно добавить как данные, если останется время.*

## Поток

```
GET  /cases                -> выбор кейса
GET  /cases/{slug}         -> вводная и вопрос
POST /cases/{slug}/answer  -> серверная валидация -> следующий вопрос ->
либо расчёт результата
GET  /cases/{slug}/result  -> баллы, пояснения, ссылка на roadmap
```

## Хранение прогресса

Для MVP подойдёт серверная сессия. Альтернатива - передача состояния через формы, но сервер всё равно должен валидировать каждый шаг. Нельзя принимать итоговый балл из браузера как достоверный. Постоянная история прохождений необязательна.

## Формат результата

- число верных или наиболее подходящих ответов;
- объяснение каждого ответа;
- краткий вывод о типе задач;
- ссылка на дорожную карту;
- пояснение, что это ознакомительный сценарий, а не профессиональная диагностика.

## Тесты кейса

- валидный ответ переводит на ожидаемый шаг;
- неизвестный вариант отклоняется;
- нельзя пропустить обязательные шаги;

- повторная отправка не начисляет баллы дважды;
- после завершения отображаются все пояснения;
- отсутствующая сессия обрабатывается безопасно.

# 10. P2P-менторство и цифровое волонтёрство

## P2P-ментор

Карточка содержит имя/псевдоним, направление, уровень обучения, темы помощи, формат и краткое описание. Форма заявки содержит имя, контакт, тему и сообщение. После валидации запись сохраняется со статусом `new`.

*В MVP не реализуются чат, календарь, автоматический подбор и реальные уведомления. На защите можно показать полный цикл: карточка → форма → подтверждение → запись в БД.*

## Цифровой волонтёр

Карточка задачи содержит название, тип организации, описание проблемы, нужные навыки, ожидаемый результат и примерный масштаб. Отклик содержит имя, контакт, навыки и сообщение.

*В первой версии нет управления сроками, назначения исполнителя и встроенной переписки. Это концепт каталога и работающий демонстрационный путь отправки отклика.*

## Общая логика заявок

Обе формы могут использовать общий сервис, но сервер обязан проверять тип заявки, существование целевого объекта и длину полей. Нельзя доверять произвольному `target_id`, присланному из браузера.

# 11. Roadmaps, блог и новости

## Структура контента

```
content/
├── roadmaps/
│   ├── frontend.md
│   ├── backend-go.md
│   └── project-manager.md
└── blog/
    ├── first-steps-in-it.md
    └── how-to-build-a-pet-project.md
```

Roadmap должен содержать цель направления, базовые знания, порядок тем, небольшие практические проекты, критерии прогресса и ссылки на бесплатные

материалы. Полноценный курс не нужен.

Для блога достаточно 2-3 материалов. Новости можно представить датированными демонстрационными публикациями со ссылками на первоисточники; автоматический импорт не требуется.

*Markdown нельзя бездумно вставлять в HTML как доверенный текст. Используйте проверенную библиотеку и безопасную конфигурацию, ограничьте произвольный HTML или санитизируйте результат.*

---

## 12. UX/UI и доступность

### Страницы

- главная;
- каталог направлений и страница направления;
- каталог кейсов, экран вопроса и экран результата;
- каталог менторов и профиль;
- каталог волонтёрских задач и карточка;
- список и страница roadmap;
- список и страница блога;
- страница «О проекте».

### Темы

Рекомендуется CSS Custom Properties:

```
:root {
   color-scheme: light;
   --color-bg: #ffffff;
   --color-surface: #f4f6f8;
   --color-text: #17202a;
   --color-muted: #52606d;
   --color-primary: #2457d6;
   --color-border: #d7dee5;
}

[data-theme="dark"] {
   color-scheme: dark;
   --color-bg: #111827;
   --color-surface: #1f2937;
   --color-text: #f9fafb;
   --color-muted: #c0cad5;
   --color-primary: #8ab4ff;
```

```
    --color-border: #465365;
}
```

*Это отправная точка, а не окончательная палитра. Настройку можно хранить в `localStorage`, а при первом посещении учитывать `prefers-color-scheme`.*

### Высокая контрастность и a11y

Отдельный атрибут вроде `data-contrast="high"` меняет CSS-переменные, делает границы и фокус заметнее и подчёркивает ссылки.

**Обязательный чек-лист:**

- клавиатурная навигация;
- видимый `:focus-visible`;
- подписи у полей форм;
- понятные сообщения об ошибках;
- логичная иерархия заголовков;
- информация не передаётся только цветом;
- `alt` у содержательных изображений;
- отсутствие мешающих анимаций;
- проверка при увеличении масштаба.

*Высокая контрастность не заменяет полноценную проверку доступности.*

---

## 13. HTTP-маршруты

| Метод | Путь | Назначение |
|---|---|---|
| GET | `/` | Главная |
| GET | `/directions` | Каталог направлений |
| GET | `/directions/{slug}` | Страница направления |
| GET | `/cases` | Каталог кейсов |
| GET | `/cases/{slug}` | Старт кейса |
| POST | `/cases/{slug}/answer` | Ответ на шаг |
| GET | `/cases/{slug}/result` | Результат |
| GET | `/mentors` | Каталог менторов |
| GET | `/mentors/{slug}` | Профиль ментора |
| POST | `/mentors/{slug}/applications` | Заявка наставнику |
| GET | `/volunteering` | Каталог задач |

| Метод | Путь | Назначение |
|---|---|---|
| GET | `/volunteering/{slug}` | Карточка задачи |
| POST | `/volunteering/{slug}/applications` | Отклик |
| GET | `/roadmaps` | Список roadmaps |
| GET | `/roadmaps/{slug}` | Дорожная карта |
| GET | `/blog` | Список статей |
| GET | `/blog/{slug}` | Статья |
| GET | `/health/live` | Проверка процесса |
| GET | `/health/ready` | Проверка готовности БД |

**Правила:**

- GET не изменяет состояние;
- формы используют POST;
- после успешного POST применяется redirect (Post/Redirect/Get);
- неизвестный ресурс возвращает 404;
- внутренние ошибки не раскрывают SQL и stack trace;
- отдельный REST API не нужен, пока единственный клиент - серверно-рендеримый сайт.

# 14. Безопасность и конфиденциальность

## Минимальные меры

- серверная валидация всех пользовательских полей;
- параметризованные SQL-запросы;
- отсутствие секретов в Git;
- переменные окружения для конфигурации;
- ограничение длины и размера запросов;
- CSRF-защита для форм при cookie-based сессиях;
- безопасный рендеринг Markdown;
- расчёт результата кейса только на сервере;
- отсутствие публичного доступа к служебным маршрутам;
- минимум собираемых персональных данных;
- публикация контактов только с согласия;
- безопасная обработка cookie и ошибок БД.

*Не нужно вводить регистрацию без необходимости. Но контактные данные в заявках могут быть персональными, поэтому их следует собирать только для оговорённой цели и не писать в логи.*

## 15. Логирование и эксплуатация

Для MVP достаточно `log/slog`. Логировать запуск, ошибки БД, внутренние ошибки, request ID и HTTP-статусы. Не логировать контактные данные, содержимое заявок, cookie и секреты.

- `/health/live` показывает, что процесс работает.
- `/health/ready` проверяет готовность приложения выполнять запросы, требующие БД.

*Prometheus, Grafana и распределённая трассировка не обязательны.*

## 16. Docker и окружение

Для демонстрации достаточно двух сервисов: Go-приложение и PostgreSQL.

### Пример `docker-compose.yml`

```yaml
services:
  web:
    build:
      context: .
    env_file:
      - .env
    ports:
      - "8080:8080"
    depends_on:
      postgres:
        condition: service_healthy

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: goit
      POSTGRES_USER: goit
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U goit -d goit"]
      interval: 5s
      timeout: 3s
      retries: 10

volumes:
  postgres_data:
```

### Пример `.env.example`

```
APP_ENV=development
HTTP_ADDR=:8080
DATABASE_URL=postgres://goit:change_me@postgres:5432/goit?sslmode=disable
POSTGRES_PASSWORD=change_me
```

*`.env` должен быть в `.gitignore`. Приведённые значения - только для локальной разработки; для production нужны отдельные секреты. Порт PostgreSQL можно не публиковать на хост, если прямой доступ не нужен.*

### Команды Makefile

```
.PHONY: run test fmt

run:
	go run ./cmd/web

test:
	go test ./...

fmt:
	gofmt -w .
```

## 17. Тестирование

### Unit-тесты

- расчёт результата кейса;
- валидация выбранного варианта;
- переход к следующему вопросу;
- валидация заявки;
- допустимые типы и статусы;
- обработка неизвестного кейса.

### HTTP-тесты

Через `net/http/httptest` проверить главную страницу, 404, некорректные формы, успешный redirect, обработку ответов кейса и безопасные сообщения об ошибках.

### Интеграционные тесты

Минимум - миграции и сохранение заявки в тестовой PostgreSQL. Если сроки ограничены, приоритетны unit- и HTTP-тесты ключевого сценария.

### Ручная проверка

Проверить узкую и широкую ширину окна, темы, клавиатурную навигацию, пустые состояния, повторную отправку формы, ошибки сети/БД и сохранность данных после перезапуска контейнеров.

## 18. План разработки на 8 недель

| Неделя | Работа | Результат |
|---|---|---|
| **1** | Аналитика, wireframes, схема БД, маршруты, репозиторий | Согласованный scope и каркас |
| **2** | Go-сервер, router, шаблоны, PostgreSQL, миграции, Docker | Запускаемая основа сайта |
| **3** | Модель кейса, шаги, серверное состояние | Прохождение кейса |
| **4** | Расчёт результата, пояснения, тесты | Завершённый основной сценарий |
| **5** | Roadmaps, Markdown, блог | Контентные страницы |
| **6** | Каталоги менторов/задач, формы, сохранение заявок | Рабочий путь подачи заявки |
| **7** | Темы, контрастность, адаптивность, интеграционные тесты | Целостный интерфейс |
| **8** | Исправления, README, схема архитектуры, репетиция | Готовая демонстрация |

**Если есть только 6 недель:**

- отказаться от админ-панели;
- сделать один полноценный кейс;
- ограничить контент двумя статьями и двумя дорожными картами;
- оставить каталоги на seed-данных;
- отложить Telegram-бота;
- сохранить тесты ключевой логики и воспроизводимый запуск.

## 19. Распределение работы

| Область | Участник A | Участник B |
|---|---|---|
| **Проектирование** | Архитектура и модель кейса | UX, wireframes, контент |
| **Backend** | Кейсы и расчёт результата | Репозитории и формы заявок |
| **Интерфейс** | Экран кейса и результата | Layout, каталоги и формы |
| **Материалы** | Тексты сценариев | Roadmaps и статьи |
| **Интеграция** | Контракты и code review | Темы и доступность |
| **Финал** | Совместные тесты и защита | Совместные тесты и защита |

*Распределение следует адаптировать под реальные навыки. Важно заранее согласовать структуры данных, маршруты и общие шаблоны.*

**Правила команды:**

- небольшие задачи с критерием готовности;
- отдельная ветка на функциональность;
- pull request перед слиянием;
- `gofmt` и тесты перед merge;
- еженедельная интеграционная демонстрация;
- фиксация архитектурных решений и известных ограничений.

## 20. Риски

| Риск | Снижение риска |
|---|---|
| Слишком широкий scope | Приоритеты Must/Should/Could |
| Конфликты при параллельной работе | Согласовать модели и маршруты заранее |
| Сложная авторизация | Не делать аккаунты без необходимости |
| Пустой контент | Подготовить минимальные материалы в начале |
| Сложность Markdown | Использовать готовую библиотеку или упростить формат |
| Необъяснимый результат кейса | Хранить правила и пояснения |
| Небезопасные формы | Валидация, параметризованный SQL, CSRF-защита |
| Недоступный интерфейс | Проверить контраст и клавиатурный фокус |
| Проект не запускается на защите | Docker Compose и smoke-тест на чистом окружении |
| Бот отнимает время | Разрабатывать только после основного сценария |
| Социальная польза выглядит абстрактной | Показать практические кейсы и взаимопомощь |

## 21. Критерии готовности MVP

- [ ] Проект запускается по README.
- [ ] Docker Compose поднимает Go-приложение и PostgreSQL.
- [ ] Главная объясняет цель и аудиторию.
- [ ] Доступен каталог направлений.
- [ ] Есть хотя бы одна дорожная карта и две статьи.

- [ ] Один кейс проходится от начала до результата.
- [ ] Результат рассчитывается сервером.
- [ ] Результат содержит пояснения и ссылку на roadmap.
- [ ] Каталоги менторов и волонтёрских задач заполнены тестовыми данными.
- [ ] Хотя бы одна форма сохраняет заявку в PostgreSQL.
- [ ] Светлая и тёмная темы работают на ключевых страницах.
- [ ] Режим повышенной контрастности реализован или честно отмечен как отложенный.
- [ ] Ключевая бизнес-логика покрыта тестами.
- [ ] В репозитории нет секретов и несанкционированных персональных данных.
- [ ] Подготовлены архитектурная схема и сценарий демонстрации.

## 22. Сценарий защиты на 5-7 минут

1. **Проблема и цель (30-45 сек.):** трудность выбора IT-направления и поиска поддержки.
2. **Главная (30 сек.):** структура платформы.
3. **Кейс (2 мин.):** выбор, ответы, серверная логика, объяснимый результат.
4. **Roadmap (45 сек.):** дальнейший путь обучения.
5. **P2P и волонтёрство (1 мин.):** каталог и отправка тестовой заявки.
6. **Доступность (30-45 сек.):** темы, контрастность, клавиатурный фокус.
7. **Техническая часть (1 мин.):** схема архитектуры, Go-пакеты, тесты и Docker Compose.

## 23. Что проект демонстрирует

- **Backend:** HTTP, слои handler/service/repository, PostgreSQL, валидация, обработка состояний, тестирование.
- **Frontend и UX:** серверный рендеринг, шаблоны, адаптивность, темы, формы и доступность.
- **Инженерная практика:** Git, code review, миграции, контейнеризация, конфигурация окружения, логирование и документация.
- **Продуктовое мышление:** аудитория, социальная проблема, сценарии, приоритизация, критерии приёмки и план развития.

## 24. Дальнейшее развитие

- Регистрация и личный кабинет.
- История прохождения кейсов и прогресс по roadmap.
- Административный интерфейс контента.
- Модерация и управление заявками.
- Telegram-бот или email-уведомления.

- Реальные консультации с согласованием времени.
- Назначение исполнителей на волонтёрские задачи.
- Пользовательское тестирование в школах и студенческих сообществах.
- Расширение библиотеки кейсов.
- Аудит доступности и аналитика без лишнего сбора персональных данных.

## 25. Итоговое позиционирование

**GO-IT: социальный код** - демонстрационная образовательная веб-платформа, объединяющая практическое знакомство с IT-профессиями, структурированные маршруты обучения и концепцию взаимопомощи.

Главный технический акцент - работающий интерактивный кейс с серверной логикой на Go. Roadmaps и блог показывают работу с контентом, P2P и цифровое волонтёрство - проектирование каталогов и форм взаимодействия, а темы и контрастность - внимание к доступности.

Проект сознательно ограничивает глубину второстепенных модулей, чтобы за 6-8 недель получить целостную, запускаемую и объяснимую систему. Критерий успеха - не количество страниц, а завершённый пользовательский сценарий, понятная архитектура, тестируемая логика и честно обозначенные границы MVP.

### Приложение. Минимальный вертикальный срез

До разработки всех страниц реализуйте один сценарий целиком:

1. Пользователь открывает `/cases`.
2. Выбирает кейс.
3. Go-сервер отображает вопрос.
4. Пользователь отправляет ответ.
5. Сервер валидирует ответ и выбирает следующий шаг.
6. Сервер рассчитывает итог.
7. Пользователь получает результат и ссылку на roadmap.
8. Unit- и HTTP-тесты проверяют переходы.

---

## 26. Заметки по улучшению (не включать в документ)

- **Безопасность Markdown:** использовать связку `goldmark` (парсинг) + `bluemonday` (санитизация) для исключения XSS.
- **Интеграционные тесты с БД:** применить `testcontainers-go` для поднятия временного Postgres-контейнера во время прогона тестов.
- **HTMX и формы:** добавить атрибуты `hx-post` и `hx-swap="outerHTML"` для обновления DOM без перезагрузки страницы.

- **Автоматическая тема:** добавить медиа-запрос `@media (prefers-color-scheme: dark)` для системного подхвата темы до первого ручного переключения.
- **Seed-данные:** написать Go-скрипт ( `go run ./cmd/seed` ), читающий JSON/MD из `content/` и делающий `INSERT` в БД при первом запуске.